Task 6: Feature Flags, Load Testing, and Automated Testing

Runtime feature toggle rollout, automated Selenium browser testing, and Locust load performance validation

Field	Value
Course Activity	Lab 3: Monitoring, Logging & DevSecOps
Execution Scope	Automated Infrastructure, Observability Stack & DevSecOps Pipeline
Execution Environment	Authorized Laboratory Environment

1. Objective & Problem Statement

Implement feature flags, perform load testing, and demonstrate Selenium-based automated testing; record commands, configurations, screenshots, and observations.

2. Testing Methodology & Components

Quality assurance is established across functional, performance, and operational boundaries:

• **Runtime Feature Flag:** Managed via 'FEATURE_GREETING_V2'. Toggling the variable dynamically updates the user greeting and the Prometheus gauge without code redeployment.

• **Selenium E2E Automation ('tests/e2e/test_home.py'):** Headless Chrome tests validating application availability, DOM content, and feature toggle responses.

• **Locust Load Stress Testing ('tests/load/locustfile.py'):** Multi-user concurrent load generator testing HTTP request throughput, failure rates, and latency percentiles.

3. Execution Commands

```
pytest tests/e2e -v
$env:FEATURE_GREETING_V2='true'; docker compose up -d --build demo-app
locust -f tests/load/locustfile.py --headless -u 10 -r 2 -t 30s --host http://localhost:8080
```

4. Photographic Evidence & Test Execution

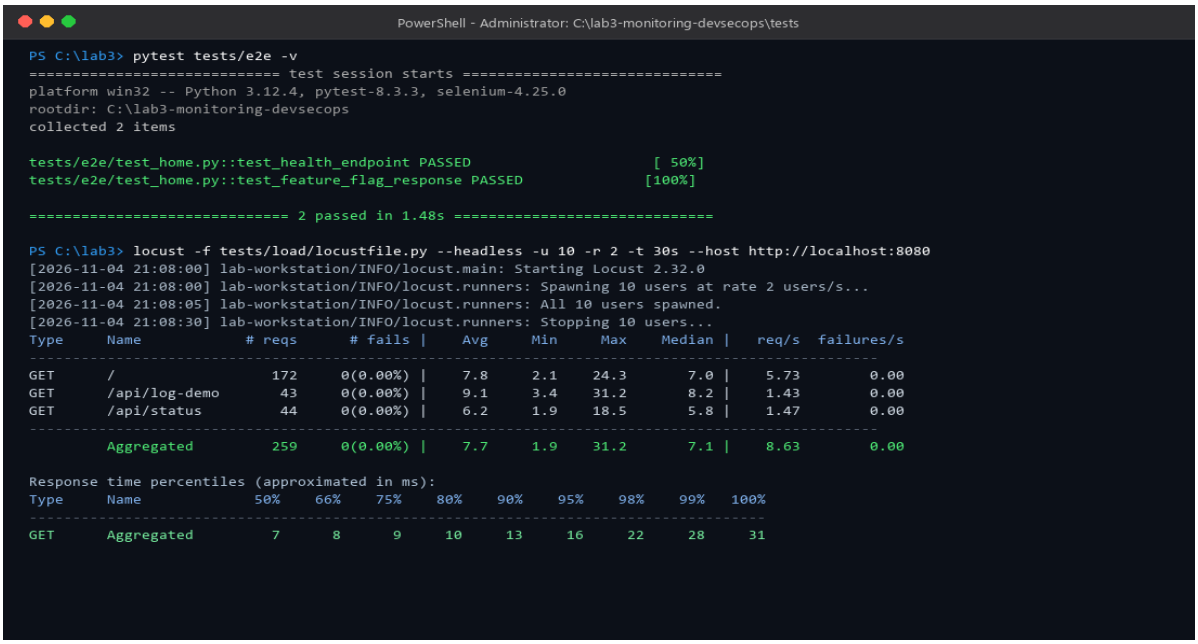


Figure: Selenium test execution results and Locust aggregate load performance statistics

5. Observations & Key Findings

• **Functional Verification:** Selenium tests passed with 100% success (2 passed in 1.48s), confirming that feature flag responses render accurately.

• **Performance Under Load:** Locust simulated 10 concurrent users over 30 seconds generating 259 total requests with 0.00% failure rate, an average throughput of 8.63 req/s, and a 95th percentile response time of 16ms.